

MCDI500

PROGRAMACIÓN PARA LA CIENCIA DE DATOS

AVANCE DEL PROYECTO FASE 3 — SEMANA 2

Nombre integrante/s:

- Karim Zaid (@karimzaid)
- Daniel Herrera (@dnietoh)
- Jeremmy Díaz (@kirijota98)
- Laura Arenas (@laura-xiphias)

Curso: MCDI500 Programación para la Ciencia de Datos · Sección 202682.1913

Docente: Dr. Omar Salinas Silva

Fecha: 30 de septiembre de 2026

Repositorio: [GitHub - laura-xiphias/proyecto-grupo3-mcdi500](#) · [GitHub](#)

ÍNDICE

II. Codificación funcional y arquitectura básica del script.....	3
III. Preprocesamiento y transformación del dataset.....	3
IV. Validación técnica y verificación del código.....	4
V. Eficiencia y optimización.....	4
Medición 1: recorrido de las carpetas.....	4
Medición 2: hora del siniestro.....	4
Medición 3: atropellos fatales por comuna.....	5
VI. Diseño estructurado del código.....	5
VII. Implementación modular / POO.....	5
VIII. Documentación de arquitectura.....	6
IX. Notebooks ejecutables.....	6
X. Repositorio GitHub.....	7
XI. Bibliografía.....	8

II. Codificación funcional y arquitectura básica del script

Organizamos el código en funciones pequeñas, cada una con una sola tarea, y las dejamos en archivos separados dentro de `F3/src/`. El notebook solo importa esas funciones y las llama en orden, igual que hicimos en la Fase 2 con `obtencion.py`, `limpieza.py` y `transformacion.py`.

Para cada cálculo escribimos dos versiones (A y B) que reciben lo mismo y devuelven lo mismo. La Tabla 1 resume qué recibe y qué devuelve cada función.

Función	Archivo	Recibe	Devuelve
<code>arbol_rglob / arbol_recursoivo</code>	<code>recorrido.py</code>	La carpeta raíz del proyecto	Lista de líneas con el árbol de carpetas
<code>hora_bucle / hora_vectorizada</code>	<code>hora.py</code>	El DataFrame de la Fase 2	Lista con la hora (0 a 23) de cada atropello
<code>fatales_filtrando / fatales_agrupando</code>	<code>busqueda.py</code>	El DataFrame y la lista de comunas	Diccionario {comuna: atropellos fatales}
<code>comparar</code>	<code>medicion.py</code>	Las dos versiones de un cálculo	Diccionario con tiempos, memoria y cuántas veces más rápida fue B

Tabla 1. Funciones del avance, con sus entradas y salidas.

La función `comparar()` es la que coordina: ejecuta las dos versiones, revisa que den lo mismo y después mide tiempo y memoria. Así cada medición del notebook se escribe en pocas líneas:

```
fila = medicion.comparar(
    "filtrar cada vez", lambda: busqueda.fatales_filtrando(df, comunas),
    "agrupar una vez", lambda: busqueda.fatales_agrupando(df, comunas),
)
```

Evitamos funciones con muchas opciones de verdadero o falso: si dos formas de hacer algo eran distintas, las escribimos como dos funciones separadas. Eso es lo que permite compararlas.

III. Preprocesamiento y transformación del dataset

Partimos del archivo procesado en la Fase 2 (`F1/data/processed/atropellos_2023_features.csv`, 5.541 filas y 44 columnas). No repetimos la limpieza ni la imputación, porque ya se evaluaron en la entrega anterior.

La única transformación nueva fue corregir la hora del siniestro. Al revisar el código de la Fase 2 encontramos que `Hora_limpia` quedó en 0 en las 5.541 filas. La causa es que se calculó desde la columna `Hora`, que en el archivo original tiene el mismo valor en todas las filas (1899/12/30 00:00, la fecha "cero" que usa Excel). La hora real está en `Hora_texto`, con valores como 19:45:00.

Con la hora corregida la variable pasa de tener 1 valor a tener 24, y se puede ver cómo se reparten los atropellos en el día: el máximo está a las 18:00 (493 casos), seguido de las 19:00 (462) y las 7:00 (386). Esta variable es importante para la pregunta del proyecto, porque la hora es parte del contexto del siniestro. No modificamos el notebook de la Fase 2 porque ya fue entregado; la corrección queda en `F3/src/hora.py`.

IV. Validación técnica y verificación del código

Comprobamos que el código funciona de tres maneras, siguiendo lo que recomienda la guía de la fase:

- **Revisión del archivo de entrada.** Antes de medir, dos `assert` confirman que el archivo tiene las 5.541 filas y los 407 casos fatales (7,35 %) que salieron en la Fase 2.
- **Mismo resultado en las dos versiones.** La función `comparar()` detiene el notebook si A y B no dan exactamente lo mismo. En las tres mediciones apareció el mensaje "Mismo resultado: OK".
- **Revisión de resultados intermedios.** Después de corregir la hora revisamos que todos los valores estén entre 0 y 23. Después de contar fatales por comuna revisamos que la suma de todas las comunas dé los mismos 407 casos del total.

```
assert df.shape[0] == 5541, "El conjunto no tiene las 5.541 filas esperadas"
assert df["Hora_corregida"].between(0, 23).all(), "Hay horas fuera del rango 0-23"
assert sum(fatales.values()) == df["Es_Fatal"].sum(), "La suma por comuna no coincide"
```

Además, el notebook muestra salidas en cada paso (tamaño de la tabla, ejemplos de `Hora_texto`, reparto por hora y las cinco comunas con más atropellos fatales), para que si algo sale mal se note en el paso donde ocurre y no al final.

V. Eficiencia y optimización

Medimos el tiempo con `timeit` y la memoria con `tracemalloc` (Python Software Foundation, s. f.-a, s. f.-b). Cada versión se ejecutó 3 veces y nos quedamos con el tiempo menor, porque los tiempos altos suelen deberse a que el computador estaba haciendo otra cosa. Todas las mediciones se hicieron sobre el conjunto completo y en el mismo equipo (Python 3.13.7 y pandas 3.0.5).

Medición	Versión A	Tiempo A	Memoria A	Versión B	Tiempo B	Memoria B	B más rápida
1 · Carpetas	rglob + filtro	0,6060 s	42,658 MB	recursiva con poda	0,0025 s	0,015 MB	245 ×
2 · Hora	bucle con <code>strptime</code>	0,0244 s	0,047 MB	<code>pd.to_datetime</code>	0,0016 s	0,183 MB	15 ×
3 · Fatales por comuna	filtrar cada vez	0,2050 s	0,173 MB	agrupar una vez	0,0008 s	0,222 MB	264 ×

Tabla 2. Resultados de las mediciones (mejor de 3 ejecuciones).

Medición 1: recorrido de las carpetas

La versión A (la que estaba en el notebook de la Fase 1) usa `rglob` para revisar todo el proyecto y al final descarta lo que no queremos mostrar. El problema es que el proyecto tiene 31.356 archivos y carpetas, y 30.934 de ellos (98,7 %) están dentro de `.venv`, así que la mayor parte del trabajo se bota. La versión B entra carpeta por carpeta y se salta `.venv` y `.git` sin revisarlas. Las dos muestran los mismos 30 elementos, pero B fue unas 245 veces más rápida y usó mucho menos memoria (0,015 MB contra 42,7 MB), porque no tiene que guardar la lista completa de rutas.

Medición 2: hora del siniestro

La versión A convierte el texto de la hora fila por fila con `strptime`; la versión B convierte la columna completa de una vez con `pd.to_datetime`, como recomienda McKinney (2022) para trabajar con pandas. B fue unas 15 veces más rápida. Lo interesante es que B usó más memoria (0,183 MB contra 0,047 MB), porque primero

arma la columna de fechas completa y después saca la hora. O sea, ser más rápida no siempre significa usar menos memoria. Aun así elegimos B, porque la diferencia de memoria es muy chica para el tamaño de nuestros datos y la diferencia de tiempo es grande.

Medición 3: atropellos fatales por comuna

Para responder la pregunta del proyecto vamos a necesitar muchas consultas por comuna. La versión A filtra la tabla completa una vez por cada una de las 284 comunas; la versión B agrupa una sola vez con `groupby` (The pandas development team, s. f.), guarda el resultado en un diccionario y después solo lo consulta. Fue la diferencia más grande: B fue unas 264 veces más rápida, con un uso de memoria parecido. Elegimos B, y es la forma que vamos a usar en la Fase 4.

En las tres mediciones la medición cambió una decisión: en la Fase 1 y 2 usábamos la versión A sin saber cuánto costaba, y ahora sabemos por qué conviene la B.

VI. Diseño estructurado del código

El código sigue un orden simple: cargar los datos, validarlos, y después hacer cada medición por separado con la misma estructura (explicación, código, interpretación). Cada medición vive en su propio archivo, así que se puede cambiar una sin tocar las otras.

Sobre la recursión. El apunte de la fase explica que la recursión se justifica cuando no se sabe de antemano cuántos niveles tiene el problema, y que cada llamada ocupa memoria según la profundidad (Universidad Andrés Bello, 2026). Con ese criterio decidimos:

- **En la tabla de atropellos no usamos recursión.** Aunque tiene una jerarquía (región, provincia y comuna), siempre son 3 niveles fijos, y un `groupby` o un bucle lo resuelve de forma más simple.
- **En las carpetas del proyecto sí la usamos.** La cantidad de niveles no es fija: cambia cada vez que se crea una carpeta y no todas las ramas tienen la misma profundidad. Un bucle obligaría a adivinar cuántos niveles anidar.

La función recursiva tiene sus dos partes: el caso recursivo (si es una carpeta, se vuelve a llamar un nivel más abajo) y el caso base (una carpeta sin subcarpetas no genera más llamadas). Como el proyecto tiene 4 niveles, la función nunca se llama más de 4 veces seguidas, muy lejos del límite de unas 1.000 llamadas que tiene Python.

```
def arbol_recursivo(carpeta, nivel=0, excluir=EXCLUIR):
    lineas = []
    for item in sorted(carpeta.iterdir()):
        if item.name in excluir:
            continue # poda
        lineas.append(" " * nivel + item.name + ("/" if item.is_dir() else ""))
        if item.is_dir():
            lineas += arbol_recursivo(item, nivel + 1, excluir) # caso recursivo
    return lineas # caso base
```

VII. Implementación modular / POO

Sacamos las funciones del notebook y las pusimos en archivos propios dentro de `F3/src/`. El notebook las importa al inicio:

```
sys.path.append("src")
import medicion, recorrido, hora, busqueda
```

Comparado con la Fase 2, lo que cambió es que ahora cada archivo tiene una sola responsabilidad y que agregamos un archivo común (`medicion.py`) para no repetir el código de medición en cada sección. En esta entrega no usamos clases, porque ninguna de estas funciones necesita recordar información entre una llamada y otra; con funciones bien separadas era suficiente.

VIII. Documentación de arquitectura

Componente	Responsabilidad	Archivo
Medición	Medir tiempo y memoria, y revisar que A y B den lo mismo	F3/src/medicion.py
Recorrido	Listar la estructura del proyecto (versión con rglob y versión recursiva)	F3/src/recorrido.py
Hora	Sacar la hora desde Hora_texto (bucle y vectorizada)	F3/src/hora.py
Búsqueda	Contar atropellos fatales por comuna (filtrando y agrupando)	F3/src/busqueda.py
Notebook	Cargar datos, validar, ejecutar las mediciones y explicar resultados	F3/F3_Nucleo_Algoritmico.ipynb

Tabla 3. Componentes del avance y dónde está cada uno.

Separamos la medición en su propio archivo porque la usan las tres comparaciones; si hubiéramos copiado ese código en cada sección, cualquier cambio (por ejemplo, medir 5 veces en vez de 3) habría que hacerlo en tres lugares. Separamos cada cálculo en un archivo distinto para poder revisar y probar uno sin tocar los demás.

Pensando en lo que viene, la función `fatales_agrupando` y la hora corregida se pueden usar directamente en la Fase 4, porque ahí vamos a necesitar muchas consultas por comuna y por hora para responder la pregunta del proyecto.

IX. Notebooks ejecutables

El entregable principal es `F3/F3_Nucleo_Algoritmico.ipynb`. Se ejecuta completo con “Restart Kernel and Run All”, sin errores, y conserva las salidas de todas las celdas, incluidas las mediciones. Antes de cada bloque de código hay una celda de texto que explica qué hace, y después de cada medición hay una interpretación.

El notebook incluye una referencia al foro técnico de la fase. En el foro conversamos sobre cuándo conviene la recursión frente a un bucle, y llegamos a que depende de si la profundidad del problema se conoce de antemano. Ese criterio es el que aplicamos en la Medición 1. También nos quedó la idea de medir antes de afirmar que una versión es mejor, que es la base de todo el notebook. Esto coincide con la recomendación de Rule et al. (2019) de explicar en el notebook no solo qué se hizo, sino por qué.

Estructura del notebook: 1. Configuración · 2. Carga y validación del conjunto de la Fase 2 · 3. Medición 1 · 4. Medición 2 · 5. Medición 3 · 6. Resumen de las mediciones · 7. Conclusiones.

X. Repositorio GitHub

El repositorio mantiene la carpeta F2/ tal como se entregó y agrega la carpeta F3/ con el notebook y los scripts:

```
F3/
├── F3_Nucleo_Algoritmico.ipynb
└── src/
    ├── __init__.py
    ├── medicion.py
    ├── recorrido.py
    ├── hora.py
    └── busqueda.py
```

Trabajamos en una rama propia (feat/f3-medicion-recursion) que se integra a main mediante pull request, igual que en la Fase 2. Seguimos la convención de commits del proyecto (feat, fix, docs, data); por ejemplo, un commit agrega el código y las mediciones y otro guarda las salidas ejecutadas del notebook. El README se actualizó con una sección de la Fase 3 que explica cómo ejecutar el notebook, qué hace cada archivo de src/, los criterios de optimización que usamos y el error de Hora_limpia. También se corrigió la versión de Python (3.13.7).

XI. Bibliografía

McKinney, W. (2022). *Python for data analysis* (3.^a ed.). O'Reilly Media.

Python Software Foundation. (s. f.-a). *timeit — Measure execution time of small code snippets*.
<https://docs.python.org/3/library/timeit.html>

Python Software Foundation. (s. f.-b). *tracemalloc — Trace memory allocations*.
<https://docs.python.org/3/library/tracemalloc.html>

Rule, A., Birmingham, A., Zuniga, C., Altintas, I., Huang, S.-C., Knight, R., Moshiri, N., Nguyen, M. H., Rosenthal, S. B., Pérez, F., & Rose, P. W. (2019). Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks. *PLOS Computational Biology*, 15(7), e1007007.
<https://doi.org/10.1371/journal.pcbi.1007007>

The pandas development team. (s. f.). *pandas documentation*. <https://pandas.pydata.org/docs/>

Universidad Andrés Bello. (2026). *Apunte de la Fase 3: Núcleo algorítmico, eficiencia y programación orientada a objetos* [Material docente, MCDI500].